

第 2 部分 深入学习 C 程序设计

这一部分包括 4 章：第 11 章预处理指令，第 12 章位运算，第 13 章常见错误分析和第 14 章 C 程序案例。这部分内容是对教材《C 程序设计(第五版)》的重要补充。要深入掌握和使用 C 语言进行编程，建议应当学习这些内容。

第 11 章 预处理指令

在《C 程序设计(第五版)》一书的程序中已用到预处理指令，如 `#include` 指令、`#define` 指令等。几乎每一个 C 程序都会包含预处理指令，它是 C 程序中的重要部分。本章对预处理指令再集中进行较系统和深入的讨论。

C 语言允许在源程序中加入一些“预处理指令”(preprocessing directive)，以改进程序设计环境，提高编程效率。这些预处理指令是由 C 标准建议的，但它不是 C 语言本身的组成部分，不能用 C 编译系统直接对它们进行编译(因为编译程序不能识别它们)。必须在对程序进行正式编译(包括词法和语法分析、代码生成、优化等)之前，先对程序中这些特殊的指令进行“预处理”(preprocess，也称“编译预处理”或“预编译”)。把预处理指令转换成相应的程序段，它们和程序中的其他部分组成真正的 C 语言程序，对预处理指令进行的预处理工作，是由称为 **C 预处理器**(preprocessor)的程序负责处理的。

在预处理阶段，预处理器把程序中的注释全部删除；对预处理指令进行处理，如把 `#include` 指令指定的头文件(如 `stdio.h`)的内容复制到 `#include` 指令处；对 `#define` 指令，进行指定的字符替换(如将程序中的符号常量用指定的字符串代替)，同时删去预处理指令。

经过预处理后的程序不再包括预处理指令，最后再由编译程序对预处理后的源程序进行实际的编译处理，得到可供执行的目标代码。现在使用的许多 C 编译系统把 C 预处理器作为 C 编译系统的一个组成部分，在编译时一气呵成。因此有的用户误认为预处理指令是 C 语言的一部分，甚至以为它们是 C 语句，这是不对的。必须正确区别预处理指令和 C 语句，区别预处理和编译，才能正确使用预处理指令。C 语言与其他高级语言的一个重要区别是可以使用预处理指令和具有预处理的功能。

C 提供的预处理功能常用的主要有以下 3 种：

- (1) 宏定义；
- (2) 文件包含；

(3) 条件编译。

这些功能分别用宏定义指令、文件包含指令和条件编译指令来实现。为了与 C 语句相区别,这些指令以符号“#”开头,指令后面没有分号。

在主教材中已经接触到一些预处理指令了(如 #include <stdio.h>, #define PI 3.14159 等),下面再作详细的介绍,善于利用预处理指令对于提高程序的质量和程序调试的效率很有好处。

11.1 宏 定 义

11.1.1 不带参数的宏定义

不带参数的宏定义是比较简单的,就是用一个指定的标识符(即名字)来代表一个字符串。它的一般形式为

#define 标识符 字符串

这就是已经介绍过的定义符号常量,例如:

```
#define PI 3.1415926
```

它的作用是:在本程序文件中用指定的标识符 PI 来代替 3.1415926 这个字符串。在进行预处理时,将程序中凡是在该指令以后出现的所有的 PI 都用 3.1415926 代替。这种方法使用户能以一个简单的名字代替一个长的字符串,因此把这个标识符(名字)称为“宏名”,在预处理时将宏名替换成字符串的过程称为“宏展开”。#define 就是宏定义指令。

【例 11.1】 输入半径,求圆周长、圆面积、圆球体积,使用不带参数的宏定义。

解: 求圆周长、圆面积、圆球体积的公式是众所皆知的。

$$\text{圆周长 } l = 2\pi r$$

$$\text{圆面积 } s = \pi r^2$$

$$\text{圆球体积 } v = \frac{4}{3}\pi r^3$$

π 的值为 3.1415926。由于 3 个公式中都用到一个常数 π ,为了避免多次重复在程序中书写 3.1415926,可以用一个标识符 PI 代替 3.1415926。

编写程序:

```
#include <stdio.h>
#define PI 3.1415926
int main()
{
    double l, s, r, v;
    printf("input radius:");
    scanf("%lf", &r);
    l = 2.0 * PI * r;
    s = PI * r * r;
    v = 4.0 / 3 * PI * r * r * r;
    printf("l = %10.4lf\ns = %10.4lf\nv = %10.4lf\n", l, s, v);
    return 0;
}
```

运行结果:

```
input radius:4
l= 25.1327
s= 50.2655
v= 268.0826
```

 **程序分析:** 定义变量 `l, s, r, v` 为 `double`(双精度)变量,以提高运算精度。如果定义为 `float` 型变量,在编译时会给出“警告”信息:“conversion from 'double' to 'float', possible loss of data”(从 `double` 型转换为 `float` 型,可能丢失数据),因为 C 语言把所有实数都作为双精度数处理。因此对第 7,8,9 行都给出“警告”。实际上程序没有错,“警告”的目的是提醒用户检查程序有无问题,如果用户检查后认为不影响结果,可以对这个“警告”不予理会。现在程序中把变量定义为 `double` 型,在编译时不会给出“警告”信息。由于变量为 `double` 型,故在输出时用 `%lf` 格式符(在 `f` 之前加小写字母 `l`),否则会输出不正确的数字。读者可以试一下。

宏名习惯用大写字母表示(程序中用 `PI`),以便与变量名相区别(但这并非规定,用小写字母并不会出错,但建议用大写)。

使用宏名代替一个字符串,可以减少程序中重复书写某些字符串的工作量。若不定义 `PI` 代表 3.1415926,则在程序中要多处出现 3.1415926,不仅麻烦,而且容易写错(或敲错)。用宏名代替,简单而不易出错,因为记住一个宏名(它的名字往往用容易理解的单词表示)要比记住一个无规律的字符串容易,而且在读程序时能立即知道它的含义,当需要改变某一个常量时,可以只改变 `#define` 指令行,一改全改。例如,定义数组大小,可以用

```
#define array_size 1000
int array[array_size];
```

先指定 `array_size` 代表常量 1000,因此数组 `array` 的大小为 1000。如果需要改变数组大小为 500,只须改 `#define` 行:

```
#define array_size 500
```

这样在程序中所有以 `array_size` 代表的 1000 都全改为 500 了。使用宏定义,可以提高程序的通用性。

 **说明:**

(1) 宏定义只是用宏名代替一个字符串,也就是只作简单的置换,不作正确性检查。如果写成

```
#define PI 3.1415926
```

即把数字 1 错写成小写字母 `l`,在预处理时也照样把字母 `l` 代入,不管是否符合用户原意,也不管含义是否有意义。预处理时不作任何语法检查。只有对已被宏展开后的源程序进行编译时才会发现语法错误并报错。

(2) 宏定义不是 C 语句,不必在行末加分号。如果加了分号则会连分号一起进行置换。例如:

```
#define PI 3.1415926;
area=PI*r*r;
```

经过宏展开后,该语句为

```
area=3.1415926; *r*r;
```

显然出现语法错误。

(3) #define 指令出现在程序中的函数的外面,宏名的有效范围为该指令行起到本源文件结束。通常,#define 指令写在文件开头,函数之前,作为文件一部分,在整个文件范围内有效。

(4) 可以用 #undef 指令终止宏定义的作用域。例如:

```
#define G 9.8
int main()
{
    :
}
#undef G
f1()
{
    :
}
```

} G 的有效范围

由于 #undef 的作用,使 G 的作用范围到 #undef 指令的前一行,因此在 f1 函数中,G 不再代表 9.8。这样可以灵活控制宏定义的作用范围。

(5) 在进行宏定义时,可以引用已定义的宏名,即可以层层置换。

【例 11.2】 在宏定义中引用已定义的宏名。

解:

```
#include <stdio.h>
#define R 3.0
#define PI 3.1415926
#define L 2*PI*R
#define S PI*R*R
int main()
{
    printf("L=%f\nS=%f\n",L,S);
    return 0;
}
```

运行结果:

```
L=18.849556
S=28.274333
```

 **程序分析:** 经过宏置换后,printf 函数中的输出项 L 被置换为 $2 * 3.1415926 * 3.0$, S 置换为 $3.1415926 * 3.0 * 3.0$,printf 语句置换为

```
printf("L=%f\nS=%f\n",2*3.1415926*3.0,3.1415926*3.0*3.0);
```

(6) 对程序中用双撇号括起来的字符串内的字符,即使与宏名相同,也不进行置换。例如例 11.2 中的 printf 函数内有两个 L 字符,一个在双撇号内,它不被宏置换,另一个在双撇号外,被宏置换。

(7) 宏定义与定义变量的含义不同,不分配存储空间。不带参数的宏定义只作简单的字符替换,千万不要把宏名当作变量名使用。

11.1.2 带参数的宏定义

带参数的宏定义不是进行简单的字符串替换,还要进行参数替换。其定义的一般形式为

#define 宏名(参数表) 字符串

字符串中包含在括号中所指定的参数。例如:

```
#define S(a,b)  a * b
:
area=S(3,2);
```

以上定义矩形面积 S,a 和 b 是边长。在程序中用了 S(3,2),把 3 和 2 分别代替宏定义中的形式参数 a 和 b,即用 $3 * 2$ 取代 (3,2)。因此赋值语句置换为

```
area=3 * 2;
```

对带参数的宏定义是这样进行展开置换的:在程序中如果有带实参的宏(如 S(3,2)),则按 #define 指令行中指定的字符串从左到右进行置换。如果串中包含宏中的形参(如 a,b),则将程序语句中相应的实参(可以是常量、变量或表达式)代替形参。如果宏定义中的字符串中的字符不是参数字符(如 a * b 中的 * 号),则保留。这样就形成了置换的字符串,见图 11.1。

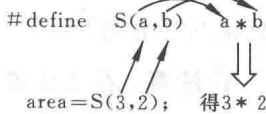


图 11.1

【例 11.3】 用带参数的宏求圆面积。

解: 解题思路: 利用带参数的宏解决问题,只要代入不同的参数,就可以得到不同的结果。比较灵活方便。

编写程序:

```
#include <stdio.h>
#define PI 3.1415926
#define S(r) PI * r * r
int main()
{ double a=3.6,area;
  area=S(a);
  printf("r=%lf\narea=%lf\n",a,area);
  return 0;
}
```

运行结果:

```
r=3.600000
area=40.715040
```

 程序分析: 赋值语句“ $\text{area}=\text{S}(\text{a});$ ”经宏置换后为

```
area=3.1415926 * a * a;
```

 说明: 对带参数的宏的展开只是将语句中的宏名后面括号内的实参字符串代替 $\# \text{define}$ 指令中的形参。本程序中的赋值语句中有 $\text{S}(\text{a})$, 在置换时, 找到 $\# \text{define}$ 指令中的 $\text{S}(\text{r})$, 将 $\text{S}(\text{a})$ 中的实参字符 a 代替宏定义中的字符串“ $\text{PI} * \text{r} * \text{r}$ ”中的形参 r , 得到 $\text{PI} * \text{a} * \text{a}$ 。

这是容易理解而且不会发生什么问题的。但是, 如果有以下语句:

```
area=S(a+b);
```

这时用实参 $\text{a}+\text{b}$ 代替 $\text{PI} * \text{r} * \text{r}$ 中的形参 r , 成为 $\text{area}=\text{PI} * \text{a}+\text{b} * \text{a}+\text{b}$; 请注意在 $\text{a}+\text{b}$ 外面没有括号, 显然这与程序设计者的原意不符。原意希望得到

```
area=PI * (a+b) * (a+b);
```

为了得到这个结果, 应当在定义宏时, 在字符串中的形式参数外面加一个括号。即

```
#define S(r) PI * (r) * (r)
```

在对 $\text{S}(\text{a}+\text{b})$ 进行宏置换时, 将 $\text{a}+\text{b}$ 代替括号中的 r , 就成了

```
PI * (a+b) * (a+b)
```

这就达到了目的。

 注意: 在定义宏时, 在宏名与带参数的括号之间不应加空格; 否则将空格以后的字符都作为替代字符串的一部分。例如, 如果有

```
#define S (r) PI * r * r //在S后有一空格
```

系统会认为 S 是符号常量(不带参数的宏名), 它代表字符串“ $(\text{r}) \text{PI} * \text{r} * \text{r}$ ”。如果在程序中有语句

```
area=S(a);
```

则被置换为

```
area=(r) PI * r * r(a);
```

显然不对了。

有些读者容易把带参数的宏和函数混淆。的确, 它们之间有类似之处, 在调用函数时也是在函数名后的括号内写实参, 也要求实参与形参的数目相等。但是带参数的宏定义与函数在本质上是不同的。主要有以下几点:

(1) 函数调用时, 先求出实参表达式的值, 然后代入形参。而使用带参数的宏只是进行字符替换。例如上面的 $\text{S}(\text{a}+\text{b})$, 在宏置换时并不求 $\text{a}+\text{b}$ 的值, 而只将实参字符“ $\text{a}+\text{b}$ ”代

替形参 r 。

(2) 函数调用是在程序运行时处理的,为形参分配临时的内存单元。而宏置换则是在预处理阶段进行的,在置换时并不分配内存单元,不进行值的传递处理,也没有“返回值”的概念。

(3) 对函数中的实参和形参都要定义类型,二者的类型要求一致,如不一致,应进行类型转换。而宏不存在类型问题,宏名无类型,它的参数也无类型,只是一个符号代表,置换时,代入指定的字符串即可。定义宏时,字符串可以是任何类型的数据。例如:

```
#define CHAR1 CHINA      (字符)
#define A 3.6             (数值)
```

CHAR1 和 A 不需要定义类型,它们不是变量,在程序中凡遇 CHAR1 均以字符 CHINA 代之;凡遇 A 均以字符 3.6 代之,显然不需定义类型。同样,对带参数的宏:

```
#define s(r) PI * r * r
```

r 也不是变量,如果在语句中有 $S(3,6)$,则置换后为 $PI * 3.6 * 3.6$,语句中并不出现 r 。当然也不必定义 r 的类型。

(4) 调用函数只可得到一个返回值,而用宏可以设法得到几个结果。

【例 11.4】 使用宏,求圆周长、圆面积和圆球体积。

解:

解题思路: 利用宏可以设法得到几个结果。

编写程序:

```
#include <stdio.h>
#define PI 3.1415926
#define CIRCLE(R,L,S,V) L=2*PI*R;S=PI*R*R;V=4.0/3.0*PI*R*R*R
int main()
{
    double r,l,s,v;
    printf("please enter r:");
    scanf("%lf",&r);
    CIRCLE(r,l,s,v);
    printf("r=%6.2lf\nl=%6.2lf\ns=%6.2lf\nv=%6.2lf\n",r,l,s,v);
    return 0;
}
```

在预处理阶段,对宏进行置换,置换后的程序如下(假设只对宏进行置换):

```
#include <stdio.h>
int main()
{
    double r,l,s,v;
    printf("please enter r:");
    scanf("%f",&r);
    l=2*3.1415926*r; s=3.1415926*r*r; v=4.0/3.0*3.1415926*r*r*r;
    printf("r=%6.2lf\nl=%6.2lf\ns=%6.2lf\nv=%6.2lf\n",r,l,s,v);
    return 0;
}
```

运行结果:

```
please enter r:3.5
r= 3.50
l= 21.99
s= 38.48
v=179.59
```

 **程序分析:** 程序中只给出一个实参 r 的值, 就可以从宏 CIRCLE 的置换中得到 3 个值 (l, s, v)。其实, 这只不过是字符代替而已, 将字符 r 代替宏定义中的 R, l 代替 L, s 代替 S, v 代替 V , 而并未在宏置换时求出 l, s, v 的值。

(5) 使用宏次数多时, 宏展开后源程序变长, 因为每展开一次都使程序增长, 而函数调用不会使源程序变长。

(6) 宏替换不占运行时间, 只占预处理时间。而函数调用则占运行时间 (分配单元、保留现场、值传递、返回)。

一般用宏来代表简短的表达式比较合适。有些问题, 用宏和函数都可以。例如:

```
#define MAX(x,y) (x)>(y)?(x):(y)
int main()
{
    int a,b,c,d,t;
    :
    t=MAX(a+b,c+d);
    :
    return 0;
}
```

赋值语句置换后为

```
t=(a+b)>(c+d)?(a+b):(c+d);
```

 **注意:** MAX 不是函数, 这里只有一个 main 函数, 在 main 函数中就能直接求出 t 的值。

这个问题也可以用函数来解决。可以定义求两个数中大者的函数 max:

```
int max(int x,int y) //定义 max 函数
{
    return(x>y? x:y);
}
```

在主函数中调用 max 函数:

```
int main()
{
    int a,b,c,d,t;
    :
    t=max(a+b,c+d); //调用 max 函数
    :
    return 0;
}
```

请分析以上两种方法。

如果善于利用宏定义, 可以实现程序的简化。例如可以事先将程序中的“输出格式”定

义好,以减少在输出语句中每次都要写出具体的输出格式的麻烦。

【例 11.5】 定义一些宏,代表不同的输出格式。

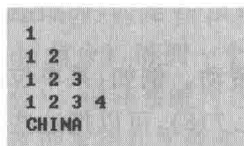
解: 解题思路: 在编写程序时,常常要写 printf 函数,针对不同的输出对象,临时编写 printf 函数中的输出格式,例如有时是输出 1 个整数,有时是输出 2 个整数,有时是输出 3 个整数,有时要输出字符串,等等,这是一件简单而重复的工作。设想:能否把常用的一些格式定义为不同的宏名,用时不必再具体地设计输出格式,而直接使用宏即可。下面分别是输出 1 个整数、2 个整数、3 个整数、4 个整数和字符串的格式声明。

编写程序:

```
#include <stdio.h>
#define PR printf
#define NL "\n"
#define D "%d"
#define D1 D NL
#define D2 D D NL
#define D3 D D D NL
#define D4 D D D D NL
#define S "%s"

int main()
{
    int a,b,c,d;
    char string[]="CHINA";
    a=1;b=2;c=3;d=4;
    PR(D1,a);           //输出 1 个整数
    PR(D2,a,b);         //输出 2 个整数
    PR(D3,a,b,c);       //输出 3 个整数
    PR(D4,a,b,c,d);     //输出 4 个整数
    PR(S,string);       //输出字符串
    return 0;
}
```

运行结果:



```
1
1 2
1 2 3
1 2 3 4
CHINA
```

程序分析: 程序中用 PR 代表 printf;以 NL 代表执行一次“换行”操作;以 D 代表输出一个整型数据的格式符“%d”;以 D1 代表输出完 1 个整数后换行;D2 代表输出 2 个整数后换行;D3 代表输出 3 个整数后换行;D4 代表输出 4 个整数后换行;以 S 代表输出一个字符串的格式符“%s\n”。可以看到,程序中写输出语句就比较简单了,只要根据需要选择已定义的输出格式即可,连 printf 都可以简写为 PR。

可以参照本例,写出各种输入输出的格式(例如单精度浮点型、双精度浮点型、长整型、十六进制整数、八进制整数、字符型等),把它们单独编成一个文件,它相当于一个“格式库”,用 #include 指令“包括”到自己所编的程序中,用户就可以根据情况各取所需了。显然在写

大程序时,这样做是很方便的。

11.2 “文件包含”处理

前面已多次用过文件包含指令了,如:

```
#include <stdio.h>
```

所谓“文件包含”处理是指一个源文件可以将另外一个源文件的全部内容包含进来,即将另外的文件内容包含到本文件之中,插入到当前的位置。C 语言用 `#include` 指令用来实现“文件包含”的操作。其一般形式为

```
#include "文件名"
```

或

```
#include <文件名>
```

图 11.2 表示“文件包含”的含义。图 11.2(a)为文件 `file1.c`,该文件中有一个 `#include <file2.c>` 指令,还有其他内容(以 A 表示)。图 11.2(b)为另一文件 `file2.c`,文件内容以 B 表示。在对预处理指令 `#include <file2.c>` 进行预处理时,系统将 `file2.c` 文件中的全部内容复制,取代 `#include <file2.c>` 指令。即把 `file2.c` 包含到 `file1.c` 中,得到图 11.2(c)所示的结果。在进行编译时,就是对经过预处理的 `file1.c`(即图 11.2(c)所示)作为一个源文件单位进行编译。

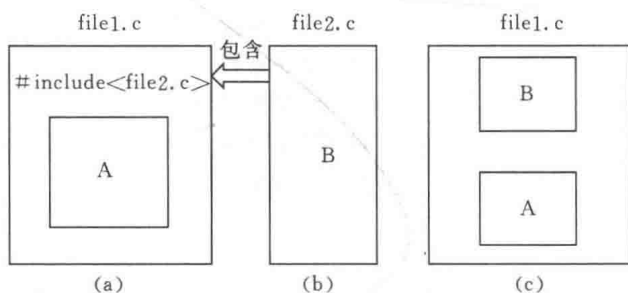


图 11.2

“文件包含”指令是很有用的,它可以节省程序设计人员的重复劳动。例如,某单位的人员往往使用一组固定的符号常量(如 $g=9.111$, $\pi=3.1415926$, $e=2.718$),可以把这些宏定义指令组成一个头文件,然后各人都可以用 `#include` 指令将这些符号常量包含到自己所写的源文件中,而不必自己重复定义这些符号常量,相当于工业上的标准零件,拿来就用。

【例 11.6】 将例 11.5 的格式宏做成头文件,把它包含在用户程序中。

解: 解题思路: 在例 11.5 中是用户自己根据需要,在程序中定义各种格式宏,但是这样用起来不方便,最好把这些格式宏作为一个头文件,供大家使用。需要用这些格式宏的用户,只须用 `#include` 指令把该文件包含到自己的程序中即可。

编写程序:

(1) 将格式宏做成头文件 `format.h`:

```
# define PR printf
# define NL "\n"
# define D "%d "
# define D1 D NL
# define D2 D D NL
# define D3 D D D NL
# define D4 D D D D NL
# define S "%s" NL
```

(2) 编写主文件 file1.c:

```
# include <stdio.h>
# include "format.h"
int main()
{
    int a,b,c,d;
    char string[]="CHINA";
    a=1;b=2;c=3;d=4;
    PR(D1,a);
    PR(D2,a,b);
    PR(D3,a,b,c);
    PR(D4,a,b,c,d);
    PR(S,string);
    return 0;
}
```

运行结果:

```
1
1 2
1 2 3
1 2 3 4
CHINA
```

 **程序分析:** 在进行编译时,并不是分别对两个文件进行编译,然后再将它们的目标程序连接的,而是在预处理时将文件 format.h 的内容包含到主文件 file1.c 中(取代 #include 指令),得到一个新的源程序(被包含的文件 format.h 中的内容成为新的源文件的一部分)。编译系统对这个文件进行编译,得到一个目标(.obj)文件。

这种常用在文件头部的被包含的文件称为“标题文件”或“头文件”,常以“.h”为后缀(h 为 header 的缩写),如“format.h”文件。当然不用“.h”为后缀,而用“.c”为后缀或者没有后缀也是可以的,但用“.h”作后缀更能表示此文件的性质。

如果需要修改程序中常用的一些参数,可以不必修改每个程序,只须把这些参数放在一个头文件中,在需要时修改头文件即可。但是应当注意,被包含文件修改后,凡包含此文件的所有文件都要全部重新编译。

在本程序中两个 #include 指令,第一个 #include 指令中的文件名用尖括号括起来(#include <stdio.h>),第 2 个 #include 指令中的文件名用双撇号括起来(#include "stdio.h")。这二者有什么区别呢?二者都合法,区别在于:用尖括号(如<stdio.h>)形式

时,系统到存放 C 库函数头文件的目录中寻找要包含的文件,这称为**标准方式**。用双撇号("file2.h")形式时,系统先在用户当前目录中寻找要包含的文件,若找不到,再按标准方式查找(即再按尖括号的方式查找)。一般来说,如果为了调用库函数而用 #include 指令来包含相关的头文件,则用尖括号,直接从存放 C 编译系统的目录中找,以节省查找时间。如果要包含的是用户自己编写的文件(这种文件一般都放在用户建立的用户当前目录中,用户编写的源程序存放在此目录中),一般用双撇号,以便到该目录中找。若该头文件没有放在用户当前目录中,则程序编写者应在双撇号内给出文件路径(如 #include "C:\wang\file2.h")。

说明:

(1) 一个 #include 指令只能指定一个被包含文件,如果要包含 n 个文件,要用 n 个 #include 指令。

(2) 如果文件 1 需要包含文件 2,而在文件 2 中又要用到文件 3 的内容,则可在文件 1 中用两个 #include 指令分别包含文件 2 和文件 3,而且文件 3 应出现在文件 2 之前,即在 file1.c 中定义:

```
#include "file3.h"
#include "file2.h"
```

由于 file3.h 的位置在 file2.h 之前,file2.h 和 file1.c 都可以用 file3.h 的内容。在 file2.h 中不必再用 #include "file3.h"了(以上是假设 file2.h 在本程序中只被 file1.c 包含,而不出现在其他场合)。

(3) 在一个被包含文件中又可以包含另一个被包含文件,即文件包含是可以嵌套的。例如,上面的问题也可以这样处理,见图 11.3。

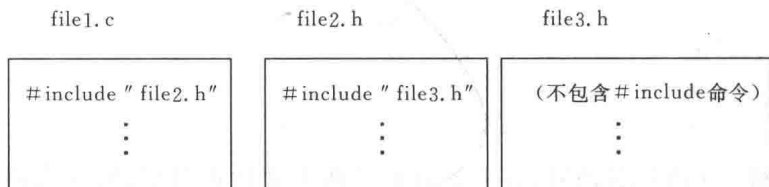


图 11.3

它的作用与图 11.4 所示相同。

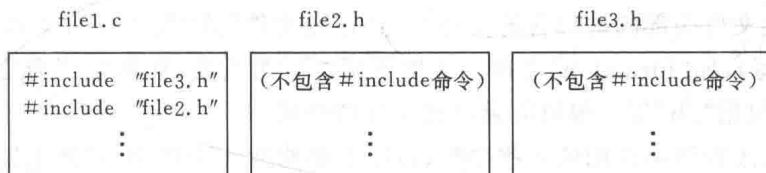


图 11.4

(4) 头文件除了可以包括函数原型和宏定义外,也可以包括结构体类型定义(见第 10 章)和全局变量定义等。

11.3 条件编译

一般情况下,源程序中所有行都参加编译。但是有时希望程序中的一部分内容只在满足一定条件时才进行编译,也就是对这一部分内容指定编译的条件,这就是“条件编译”(condition compiling)。有时,希望在满足某条件时对某一组语句进行编译,而当条件不满足时则编译另一组语句。

条件编译指令有以下几种形式:

(1) #ifdef 标识符

程序段 1

#else

程序段 2

#endif

它的作用是:若所指定的标识符已经被 #define 指令定义过,则在程序编译阶段对程序段 1 进行编译;否则编译程序段 2。实际上,预处理器是这样执行的,当发现所指定的标识符已经被 #define 指令定义过,就保留源程序中的程序段 1,而忽略程序段 2(把程序段 2 删除),否则就保留程序段 2,而忽略程序段 1。即在最后提供编译的源程序中只包括程序段 1,或只包括程序段 2。

条件编译指令中的 #else 部分可以没有,即

#ifdef 标识符

程序段

#endif

这里的“程序段”可以是语句组,也可以是指令行。这种条件编译对于提高 C 源程序的通用性是很有好处的。如果一个 C 源程序在不同计算机系统上运行,而不同的计算机又有一定的差异(例如,有的机器以 16 位(2 个字节)来存放一个整数,而有的则以 32 位存放一个整数),这样在不同的计算机上编译程序时往往需要对源程序作必要的修改,这就降低了程序的通用性。可以用以下的条件编译来处理:

```
#ifdef COMPUTER_A           //如果已定义过 COMPUTER_A
    #define INTEGER 16      //编译此指令行
# else                       //否则
    #define INTEGER_SIZE 32 //编译此指令行
#endif
```

如果在这组条件编译指令之前曾出现以下指令行:

```
#define COMPUTER_A 0
```

或将 COMPUTER_A 定义为任何字符串,甚至是

```
#define COMPUTER_A
```

即只要 COMPUTER_A 已被定义过,则在程序预编译时就会包括下面的指令行:

```
#define INTEGER_SIZE 16
```

否则,就对下面的指令行进行预编译:

```
#define INTEGER_SIZE 32
```

则预编译后程序中的 `INTEGER_SIZE` 都用 16 代替;否则都用 32 代替。

这样,源程序可以不必做任何修改就可以用于不同类型的计算机系统。当然以上介绍的只是一种简单的情况,读者可以根据此思路设计出其他条件编译。

例如,在调试程序时,常常希望输出一些所需的信息,而在调试完成后不再输出这些信息。可以在源程序中插入以下的条件编译段:

```
#ifdef DEBUG
    printf("x=%d,y=%d,z=%d\n",x,y,z);
#endif
```

如果在它的前面有以下指令:

```
#define DEBUG
```

则在程序运行时输出 `x,y,z` 的值,以便调试时分析。调试完成后只须将这个 `#define` 指令删去即可。有人可能觉得不用条件编译也可达此目的,即在调试时加一批 `printf` 语句,调试后一一将 `printf` 语句删去。的确,这是可以的。但是,当调试时加的 `printf` 语句比较多时,修改的工作量是很大的。用条件编译,则不必一一删改 `printf` 语句,只须删除前面的一条“`#define DEBUG`”指令即可,这时所有的用 `DEBUG` 作标识符的条件编译段都使其中的 `printf` 语句不起作用,即起统一控制的作用,如同一个“开关”一样。

(2) `#ifndef` 标识符

程序段 1

```
#else
```

程序段 2

```
#endif
```

同第 1 种形式相比,只是第 1 行不同:将 `ifdef` 改为 `ifndef`。它的作用是:若指定的标识符未被定义过,则编译程序段 1;否则编译程序段 2。这种形式与第 1 种形式的作用相反。

以上两种形式的用法差不多,根据需要任选一种,视方便而定。例如,上面调试时输出信息的条件编译段也可以改为

```
#ifndef RUN
    printf("x=%d,y=%d,z=%d\n",x,y,z);
#endif
```

如果在此之前未对 `RUN` 定义,则输出 `x,y,z` 的值。调试完成后,在运行之前,加以下指令:

```
#define RUN
```

再进行编译和运行,则不会输出 `x,y,z` 的值。

(3) #if 表达式

程序段 1

#else

程序段 2

#endif

它的作用是当指定的表达式值为真(非零)时就编译程序段 1;否则编译程序段 2。可以事先给定条件,使程序在不同的条件下执行不同的功能。

【例 11.7】输入一行字母字符,根据需要设置条件编译,使之能将字母全改为大写输出,或全改为小写字母输出。

解: 解题思路: 用一个字符数组存放一行字符,其中包括大写字母和小写字母。题目要求根据用户指定,把所有字母全改为大写字母,或者全改为小写字母。可以用条件编译来处理: 定义一个宏 LETTER,用“#if LETTER”指令检测,如果 LETTER 代表 1(真),就编译一组语句,把小写字母改为大写字母;如果 LETTER 代表 0(假),就编译另一组语句,把大写字母改为小写字母。

编写程序:

```
#include <stdio.h>
#define LETTER 1 //定义宏 LETTER 代表 1
int main()
{
    char str[20]="C Language",c;
    int i=0;
    while((c=str[i])!='\0') //当前字符不是'\0'时
    {
        #if LETTER //条件编译开始,如果 LETTER 为真(1)
            if(c>='a' && c<='z') //若当前字符为小写字母
                c=c-32; //改为大写字母
        #else //如果 LETTER 为假(0)
            if(c>='A' && c<='Z') //若当前字符为大写字母
                c=c+32; //改为小写字母
        #endif //条件编译结束
        printf("%c",c); //输出此字符
        i++; //指向下一个字符
    }
    printf("\n");
    return 0;
}
```

运行结果:

C LANGUAGE

 **程序分析:** 现在先定义 LETTER 为 1,这样在对条件编译指令进行预处理时,由于 LETTER 为真(1),就将第 1 个 if 语句保留而删除第 2 个 if 语句,这样经编译后运行,就会使小写字母变为大写字母。如果将程序第 2 行改为

```
#define LETTER 0
```

则在预处理时,保留第 2 个 if 语句,而删除第 1 个 if 语句。经过编译后运行,就会使大写字母变成小写字母(大写字母与相应的小写字母的 ASCII 代码差为 32)。此时运行结果为

C language

有的读者可能会问,不用条件编译指令而直接用 if 语句也能达到要求,用条件编译指令有什么好处呢?的确,对这个问题完全可以不用条件编译处理而用 if 语句处理,但那样做,目标程序长(因为所有语句都编译),运行时间长(因为在程序运行时要对 if 语句进行测试)。而采用条件编译,可以减少被编译的语句,从而减少目标程序的长度,减少运行时间。当条件编译段比较多时,目标程序长度可以大大减少。以上举例是最简单的情况,只是为了说明怎样使用条件编译,有人会觉得其优越性不太明显,如果程序比较复杂而善于使用条件编译,其优越性是比较明显的。

本章介绍的预处理功能是 C 语言特有的,有利于程序的可移植性,增加程序的灵活性。在学习深入并编写较大的程序时,善于利用预处理指令,对提高程序的质量会有好处的。